

I. UN EXEMPLE



On utilise une base de données « **Tourisme** » constituée de quatre relations :

Relation **Stations**

Nom_Station	Capacite	Lieu	Region	Tarif
Tanger	350	Maroc	Afrique	1200
La Bourboule	250	Auvergne	Europe	700
Victoria	200	Seychelles	Ocean Indien	1500
Courchevel	400	Alpes	Europe	2200

Relation **Activités**

Libelle	Nom_Station	Prix
Pêche	La Bourboule	50
Randonnée	La Bourboule	0
Plongée	Tanger	120
Plongée	Victoria	130
Ski	Courchevel	120
Excursion	Tanger	60

Relation **Clients**

Id	Nom	Prenom	Ville	Region	Solde
1	Bauer	Elmut	Berlin	Europe	9825
2	Smith	John	Londres	Europe	12436
3	Johnson	Britney	New York	Amérique	6721

Relation **Séjours**

Id_client	Station	Arrivee	Nb_Places
1	Courchevel	17/02/2020	2
3	Tanger	17/11/2019	5
2	Courchevel	28/01/2019	4
3	La Bourboule	20/07/2017	3
3	Victoria	13/09/2016	6
2	La Bourboule	13/08/2019	3
3	Courchevel	27/02/2018	5
1	Victoria	05/09/2018	3



..\EXEMPLE Tourisme\create tourisme.txt

II. CREER UNE TABLE

La création de la table précise les attributs et leur domaine, le type de clés éventuelles et les contraintes utilisateurs éventuelles.

1. TABLE DES STATIONS

```
CREATE TABLE Stations(
    Nom_Station VARCHAR(255) PRIMARY KEY,
    Capacite INTEGER,
    Lieu VARCHAR(255),
    Region VARCHAR(255),
    Tarif INTEGER);
```

a) Clé primaire – clé étrangère

👉 Ici, le nom de la station est donc **déclaré clé primaire**. Le SGBD s'assure donc que chaque station ne peut faire l'objet que d'un enregistrement unique.

- Pour utiliser **plusieurs** attributs comme clé primaire, on spécifie la contrainte après les attributs :

```
CREATE TABLE Stations(
    Nom_Station VARCHAR(255),
    Capacite INTEGER,
    Lieu VARCHAR(255),
    Region VARCHAR(255),
    Tarif INTEGER,
    PRIMARY KEY(Nom_Station) );
```

👉 La déclaration de clé étrangère se fait en utilisant le mot clé REFERENCES suivi de la table où se trouve la clé primaire et de son nom :

```
CREATE TABLE Activites(
    Libelle VARCHAR(255),
    Nom_Station VARCHAR(255) REFERENCES Stations(Nom_Station),
    Prix INTEGER);
```

b) Types numériques

👉 Le standard SQL définit plusieurs types pour les données numériques, parmi lesquels les suivants :

nom du type	description
INTEGER	entier 32 bits signé
INT	alias pour INTEGER
SMALLINT	entier 16 bits signé
BIGINT	entier 64 bits signé
DECIMAL(t, f)	décimal signé de t chiffres dont f après la virgule
REAL	flottant 32 bits (valeur approchée)
DOUBLE PRECISION	flottant 64 bits (valeur approchée)

Exemple 1 Le type **DECIMAL(5, 2)** permet de stocker des valeurs décimales de 5 chiffres dont 2 après la virgule, soit des valeurs exactes entre -999,99 et 999,99 (format adapté aux données financières)

c) Types textes

👉 Le standard SQL définit plusieurs types pour les textes, plus ou moins bien supportés suivant le SGBD utilisé. On en donne ici trois parmi les plus communs :

nom du type	description
TEXT	chaîne de taille quelconque
VARCHAR(n)	chaîne d'au plus n caractères
CHAR(n)	chaîne d'exactly n caractères. Les caractères manquants sont complétés par des espaces

2. TABLES DES SEJOURS

```
CREATE TABLE Sejours(
    Id_Client INTEGER,
    Station VARCHAR(255),
    Arrivee DATE NOT NULL,
    Nb_Places INTEGER,
    CHECK(Nb_Places>0));
```

d) Type des dates et des durées

nom du type	description
DATE	format 'AAAA-MM-JJ '
TIME	format 'hh :mm :ss'

☞ **DATE NOT NULL** impose à l'utilisateur de saisir une date (l'absence de date c'est-à-dire la date NULL sera rejetée par le SGBD)

e) Contraintes utilisateur

☞ **CHECK(Nb_Places>0)** provoque la vérification à chaque mise à jour de cette contrainte d'un nombre de participants positif.

3. SUPPRIMER UNE TABLE

C'est le mot réservé **DROP** qui permet la suppression d'une table si elle ne sert pas de référence pour une clé étrangère dans une autre table

Exemple 2 La requête **DROP TABLE Sejours** supprime la table et toutes les données qui y figurent.

Mais la requête **DROP TABLE Stations** génère une erreur du SGBD chargé de garantir la cohérence des données (la table Activités **REFERENCES Stations(Nom_Station)** !)

III. REQUETES D'INTERROGATION

Les requêtes d'interrogation visent à *sélectionner* de manière plus ou moins fine des données dans la base. Elles utilisent le mot réservé **SELECT**.

1. AVEC SELECT MAIS SANS SELECTION

Pour afficher **tous les enregistrements** d'une table on utilise :

SELECT * FROM tablename;

SELECT * FROM stations

"Tanger"	350	"Maroc"	"Afrique"	1200
"La Bourboule"	250	"Auvergne"	"Europe"	700
"Victoria"	200	"Seychelles"	"Ocean Indien"	1500
"Courchevel"	400	"Alpes"	"Europe"	2200

2. SELECTIONNER LES COLONNES

Pour afficher des colonnes d'une table on utilise:

SELECT col1, col2 FROM table_name ;

SELECT Nom_Station, Lieu FROM stations

"Tanger"	"Maroc"
"La Bourboule"	"Auvergne"
"Victoria"	"Seychelles"
"Courchevel"	"Alpes"

3. SELECTIONNER SANS DOUBLONS



La spécification des clés permet d'éviter les doublons dans les tables, mais la requête de sélection peut donner lieu à des données en double :

Exemple 3 `SELECT Libelle FROM Activites`



Résultat attendu :

Pour éviter d'obtenir deux lignes identiques en réponse à la requête **SELECT**, on utilise le mot-clé **DISTINCT**

`SELECT DISTINCT Libelle FROM Activites`



Résultat attendu :

4. SELECTIONNER EN TRIANT



On peut trier la réponse à une requête avec la clause `ORDER BY` suivie de la liste des attributs et du mot-clé `ASC` pour un tri croissant (tri par défaut) ou du mot-clé `DESC` pour un tri décroissant.

Exemple 4 `SELECT Nom_Station,Capacite FROM Stations ORDER BY Capacite ASC`



Résultat attendu :

- Pour du texte, l'ordre croissant est l'ordre alphabétique.

Exemple 5 `SELECT Nom_Station,Tarif,Region
FROM Stations
ORDER BY Region ASC , Tarif DESC`



Résultat attendu :

5. AFFINER LA SELECTION AVEC WHERE
a) Sélection ciblée

Exemple 6 SELECT Nom_Station,Lieu FROM stations WHERE region='Europe'



Résultat Attendu :

La clause **WHERE** spécifie une condition booléenne, qui utilise les mots clés de logique booléenne (**AND**, **OR**, **NOT**) et les opérateurs de comparaison (**<**, **<=**, **>**, **>=**, **=** et **<>**)

Exemple 7 SELECT Nom_Station,Tarif,Lieu
FROM Stations
WHERE Capacite>300
ORDER BY Tarif DESC



Résultat Attendu :

b) Sélection floue


Pour sélectionner les noms des clients dont le nom commence par « B » :

SELECT Nom FROM Clients WHERE Nom LIKE 'B%'

"Bauer"

- Les modèles de recherche sont multiples :
- Recherche de toutes les valeurs de l'attribut qui se terminent par « B » : **WHERE Nom LIKEB**
- Recherche de toutes les valeurs de l'attribut qui contiennent « B » : **WHERE Nom LIKE %B%**
- Recherche de toutes les valeurs de l'attribut qui commencent par « B » et qui se terminent par « on » : **WHERE Nom LIKE B%on**

6. SELECTIONNER SUR PLUSIEURS TABLES (JOINTURE)

Exemple 8 On souhaite obtenir le nom des clients avec le nom des stations où ils ont séjourné.
Tables concernées :

Relation Clients

Id	Nom	Prenom	Ville	Region	Solde
1	Bauer	Elmut	Berlin	Europe	9825
2	Smith	John	Londres	Europe	12436
3	Johnson	Britney	New York	Amérique	6721

Relation Séjours

Id_Client	Station	Arrivee	Nb_Places
-----------	---------	---------	-----------

Exo 2 : Déterminer une requête SQL qui permet d'obtenir les stations, leurs lieux et leurs régions, le nom des clients qui y ont séjourné, ainsi que le prix déboursé, dans l'ordre croissant du prix du séjour.

SELECT ...

7. FONCTIONS D'AGREGATION

Les fonctions dites « d'agrégation » permettent d'effectuer des opérations statistiques sur une colonne (en général de type numérique).

On peut citer parmi les plus courantes les fonctions suivantes :

-  COUNT() qui permet de décompter les enregistrements sur une table ou une colonne ;
-  MAX() et MIN() qui permettent d'obtenir la valeur max ou min d'une colonne sur un ensemble de lignes;
-  SUM() et AVG() qui permettent d'obtenir la somme ou la moyenne des valeurs d'une colonne sur un ensemble de ligne ;

Le retour de ces fonctions est une table à une seule case nommée dans la requête avec le mot-clé AS :

Exemple 9 Nombre de stations en Europe dans la Base Tourisme :

SELECT COUNT(Nom_Station) AS Nombre FROM Stations WHERE Region='Europe'



Résultat Attendu :

Exemple 10 Tarifs minimum, maximum et moyen des stations :

**SELECT MIN(Tarif)AS mini, MAX(Tarif) AS maxi, AVG(Tarif) AS moyen
FROM Stations**



Résultat Attendu :

Exemple 11 Nombre total de places réservées par M. Smith pour l'ensemble des séjours :

**SELECT SUM(Nb_Places) AS resa_smith
FROM Sejours JOIN Clients ON Sejours.Id_Client=Clients.Id
WHERE Nom='Smith'**

Ou bien :

**SELECT SUM(Nb_Places) AS resa_smith
FROM Sejours JOIN Clients ON Sejours.Id_Client=Clients.Id AND Nom='Smith'**



Résultat Attendu :

- Le même résultat peut être obtenu en enchainant les requêtes de sélection

```
SELECT SUM(Nb_Places) AS resa_smith
FROM (SELECT Nb_Places, Id_Client, Id, Nom FROM Sejours ,Clients) AS tmp
WHERE (tmp.Id_Client=tmp.Id AND Nom='Smith')
```



Création d'une table intermédiaire avec SELECT NB_Places, Id_Client, Id, Nom FROM Sejours ,Clients, renommée tmp dans laquelle se fait la deuxième sélection avec la clause WHERE :

Nb_Places	Id_Clien	Id	Nom
2	1	1	"Bauer"
2	1	2	"Smith"
2	1	3	"Johnson"
5	3	1	"Bauer"
5	3	2	"Smith"
5	3	3	"Johnson"
4	2	1	"Bauer"
4	2	2	"Smith"
4	2	3	"Johnson"
3	3	1	"Bauer"
3	3	2	"Smith"
3	3	3	"Johnson"
6	3	1	"Bauer"
6	3	2	"Smith"
6	3	3	"Johnson"
3	2	1	"Bauer"
3	2	2	"Smith"
3	2	3	"Johnson"
5	3	1	"Bauer"
5	3	2	"Smith"
5	3	3	"Johnson"
3	1	1	"Bauer"
3	1	2	"Smith"
3	1	3	"Johnson"

IV. Exos SELECTION

1. ENCORE DU TOURISME

Exo 3 : Donner l'expression SQL des requêtes permettant d'obtenir les données suivantes ainsi que le résultat attendu :

- Noms de stations de plus de 200 places.
- Noms des clients dont le nom commence par 'J' ou dont le solde est supérieur à 10 000.
- Nom des stations qui proposent de la plongée.

Exo 4 : Donner l'expression SQL des requêtes permettant d'obtenir les données suivantes ainsi que le résultat attendu :

- Noms des clients qui sont allés à La Bourboule.
- Nom des stations visitées par des Européens.

Exo 5 : Donner l'expression SQL des requêtes permettant d'obtenir les données suivantes ainsi que le résultat attendu :

- Nombre de séjours ayant eu lieu à Victoria. (résultat dans une colonne nommée 'Total').

b. Prix moyen d'une activité à Tanger. (résultat dans une colonne nommée 'prix_moyen_activités_Tanger')

2. DES ETUDES

Exo 6 : En ligne ici : <https://eric.univ-lyon2.fr/jdarmont/tutoriel-sql/>

3. D'AUTRES SITUATIONS

[TD-BD-1A.PDF](#)

4. DE LA LECTURE

Exo 7 : On travaille sur la base de données d'une médiathèque.

[TD Médiathèque\TD BD Médiathèque.docx](#)

V. INSERTION ET MISE A JOUR

1. INSERTION DE DONNEES



Un nouveau touriste (chinois, pekinois) se présente :

INSERT INTO Clients

VALUES(4, 'Yuan', 'Pong', 'Pekin', 'Chine', 0)

Pour insérer un enregistrement dans une table on utilise:

INSERT INTO table_name VALUES(attribut1,attribut2,...) ;



Deux autres touristes se présentent. La seule requête suivante suffira à les intégrer dans la table :

INSERT INTO Clients

VALUES(5,'De Oliveira','Manuel','Porto','Europe',0),(6,'Sako','Abdu','Abidjan','Afrique',0)

- Contrôles effectués par le SGBD lors de cette requête :

- On peut insérer les attributs dans un ordre quelconque à condition de le préciser avec la table :

INSERT INTO Clients(Id, Prenom, Nom, Region, Ville, Solde)

VALUES(7,'Ping', 'Yuan', 'Chine', 'Pekin', 0)

2. MODIFICATION DE DONNEES



Le client John Smith fait un règlement partiel. Son solde est mis à jour :

UPDATE Clients SET Solde=11564 WHERE Prenom='John' AND Nom='Smith'

- Opération **risquée** sauf si l'on est certain que John Smith est unique dans la table des clients...

Pour modifier un enregistrement existant vérifiant éventuellement) une condition booléenne c,

UPDATE table_name SET attribut1=valeur1 [WHERE c]

Exemple 12 Quel sera l'effet de la requête suivante :

UPDATE Activites SET Libelle=UPPER(Libelle), Prix=1.1*Prix

3. SUPPRESSION DE DONNEES

Pour supprimer un enregistrement dans une table vérifiant une condition booléenne c on utilise:

DELETE FROM table_name WHERE c ;



On doit donc alors fournir au minimum le nom de la table sur laquelle va s'effectuer la suppression et être plutôt confiant dans l'expression de la condition c .

Exemple 13 Quel sera l'effet de la requête suivante :

DELETE FROM Clients WHERE id $\geq$ 4

VI. Exos

1. ENCORE DU TOURISME

Exo 8 : Madame Kariboo Juliette en touriste.

- Donner l'expression SQL des requêtes permettant d'ajouter la cliente venant de Toronto : Mme Kariboo Juliette avec un solde de 7213 €. Mme Kariboo a séjourné avec sa fille et son mari à la Bourboule le 10/03/2019
- Peut-on dans l'état ajouter à cette base le fait que Mme Kariboo a fait de la randonnée.

Exo 9 : Le centre de la station de Courchevel augmente sa capacité de 50 places et ses tarifs de 3%.

- Donner l'expression SQL de la requête qui permettra de mettre à jour la base de données.
- Peut-on changer ici le nom de l'attribut « prix » en « prix HT » dans la relation Activités par une requête de type UPDATE ?

Exo 10 : Mme Kariboo Juliette se désengage.

- Donner l'expression SQL de la requête qui permettra de la faire disparaître de la base.
On suppose que la structure est bien correcte, à savoir que l'attribut Id_Client de la relation Sejours est bien une clé étrangère liée en référence à l'attribut Id de la relation Clients et que la création de la clé étrangère de la table a spécifié la clause ON DELETE CASCADE.
- Que faire si cette clause n'a pas été spécifiée ?